z

TE3001B.101 - Fundamentación de robótica (Gpo 101)

Control

Robotics Control Lab 4.2

Team Funcionaba ayer

David Alejandro Soni Cuevas	A01571777
Abraham de Jesús Maldonado Mata	A00838581
David Gilberto Lomelí Leal	A01571193
Irvin David Ornelas García	A00839065

Professor:
Nezih Nieto Gutierrez

1. Abstract

A Cartesian PD controller was implemented for the xArm Lite 6 using MoveIt Servo. The system was evaluated under baseline, sinusoidal, and Gaussian perturbations. Results show increased tracking error under stochastic noise, particularly due to derivative sensitivity to high-frequency components. The controller remained stable across all experiments with fixed gains.

2. System Overview

2.1. Robot Setup

The experiments were conducted using the xArm Lite 6, controlled through MoveIt Servo within the ROS 2 framework. MoveIt Servo enables real-time Cartesian velocity control by converting **TwistStamped** commands into joint velocity commands while enforcing joint limits and safety constraints.

The Cartesian position error is computed using TF2, which provides the transformation between the base frame (**link_base**) and the end-effector frame (**link_eef**). All experiments were executed using the required real-robot launch configuration, ensuring consistent hardware and servo initialization across trials. The robot operated within a predefined safe Cartesian workspace to avoid singularities and mechanical limits.

All experiments were executed with the following required launch command:

```shell
ros2 launch xarm_moveit_servo lite6_moveit_servo_realmove.launch.py
robot_ip:=192.168.1.123
```

The robot operated within a constrained and safe Cartesian workspace to prevent singularities and mechanical limits.

2.2. Software Architecture

The **trajectory generator** produces a smooth, continuous Cartesian reference position as a function of time.

The **position controller** computes the Cartesian error between the desired and actual end-effector pose (obtained from TF) and generates a velocity command using a PD control law. The controller includes deadband and velocity saturation to ensure stability and safe real-robot operation.

The **perturbation injector** optionally adds deterministic (sinusoidal) or stochastic (Gaussian) disturbances to the commanded velocity to evaluate disturbance rejection and noise sensitivity.

Finally, **MoveIt Servo** converts the Cartesian velocity command into joint velocity commands while enforcing joint limits, velocity constraints, and collision avoidance before sending them to the physical robot.

This modular structure enables controlled experimentation while keeping controller gains fixed across all test conditions.

3. Trajectory Design

3.1 Mathematical Definition

The chosen trajectory is a parameterized Lissajous curve (figure-eight or infinite), designed to continuously excite multiple axes simultaneously, providing a more rigorous tracking challenge than a standard circle. The trajectory in the xy-plane is defined by the following parametric equations:

$$x(t) = c_x + S(t)Asin(2\pi ft)$$

$$y(t) = c_y + S(t)\frac{A}{2}sin(4\pi ft)$$

Where cx and cy represent the initial end-effector coordinates (center of the trajectory), A=0.15 m is the amplitude (radius), and f is the operational frequency.

3.2 Soft Start Stratery

To prevent infinite jerk and immediate velocity saturation upon initialization, a linear time-scaling function S(t) was introduced. This acts as a soft-start multiplier that linearly ramps the trajectory amplitude from 0 to 1 over the first 2.0 seconds of operation:

$$S(t) = min(max(\frac{t}{2.0}, 0.0), 1.0)$$

4. Controller Design

4.1 Control Law

A Cartesian Proportional-Derivative (PD) controller computes the commanded velocity twist vc(t) sent to MoveIt Servo. The control law in the task space is:

$$V_c(t) = K_p e(t) + K_d \frac{de(t)}{dt}$$

Where Kp and Kd are 3×3 diagonal gain matrices. Basically, this control acts on the x, y, z axis.

4.2 Error Definition

The feedback error is the spatial difference between the desired trajectory pose and the actual end-effector pose extracted via TF2 (**link_base** to **link_eef**):

$$e(t) = X_{des}(t) - Xact$$

4.3 Deadband Implementation

Mechanical backlash and sensor noise can cause control chattering when the error is negligibly small. A deadband ϵ=0.002 m (2 mm) was implemented per axis. If $|e_i(t)| < \epsilon$, the commanded velocity for that axis is forced to zero, preventing micro-oscillations.

4.4 Velocity Saturation

To adhere to hardware safety constraints and prevent emergency halts triggered by the xArm controller, the final velocity command is clamped to a maximum threshold (Vmax=0.15 m/s) before publishing:

$$v_{c,i} = clip(V_{c,i,} - V_{max}, V_{max})$$

4.5 Gain Selection and Stability

Gains were empirically tuned using an automated grid-search script on the physical robot. Instead of evaluating solely based on tracking error, a custom scoring function was developed to balance tracking accuracy with control effort and hardware safety. The objective function minimized the following score:

$$Score = RMSE_{total} + \lambda_{sat} \cdot R_{sat}$$

The penalty weight, $\lambda_{sat} = 0.10$, and R_{sat}, the ratio of cycles where the velocity reached the saturation limit, were utilized in the optimization. Initial simulations yielded optimal gains of Kp=2.625 and Kd=0.625. Using these values as a baseline, the process was replicated on the physical robotic arm. This systematic approach ensured that the selected gains (Kp=2.665, Kd=0.64) minimized the tracking error without causing continuous motor saturation.

4.6 Velocity Low-Pass Filtering

To mitigate the inherent noise amplification caused by the derivative term (Kd), especially under stochastic perturbations (Gaussian noise), we do an experiment with a first-order exponential moving average (EMA) low-pass filter was applied to the commanded velocity.

5. Experimental Setup

5.1 Baseline

The robot tracked the trajectory under nominal conditions. The controller outputted Twist commands directly to MoveIt Servo without external interference to establish the minimum achievable RMSE

5.2 Sinusoidal Pertubation

A deterministic perturbation was injected into the velocity command pipeline. The injector added a continuous sine wave to the X-axis velocity to evaluate the controller's capacity to reject low-frequency, predictable disturbances.

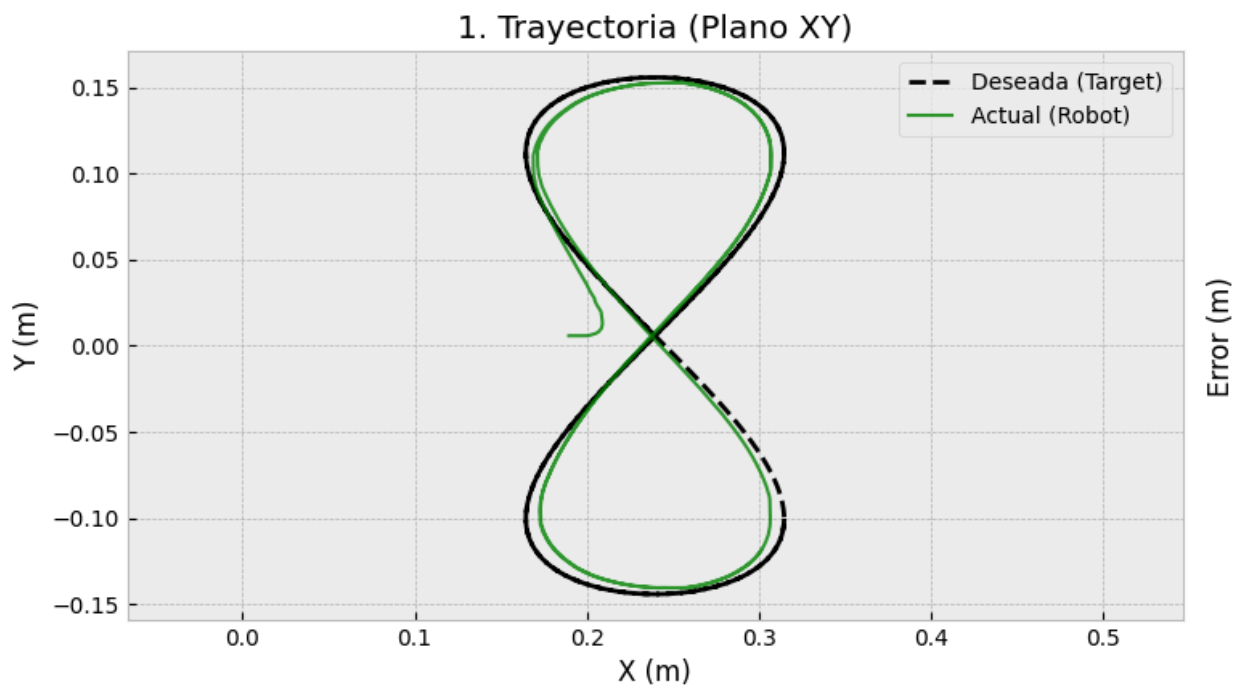
5.3 Gaussian Perturbation

A stochastic perturbation was introduced using Gaussian white noise. A standard deviation was carefully selected to prevent hardware lockouts while injecting high-frequency, unpredictable disturbances into the control loop.

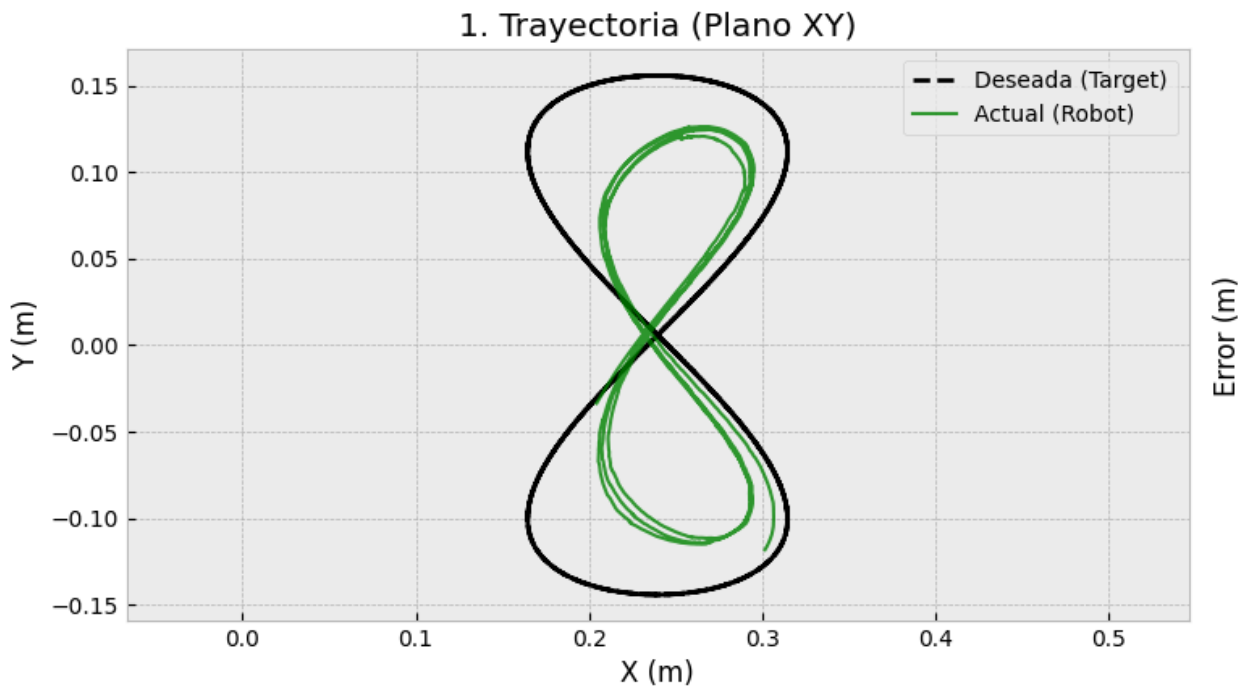
6. Results

6.1 Desired vs Actual Position Plot

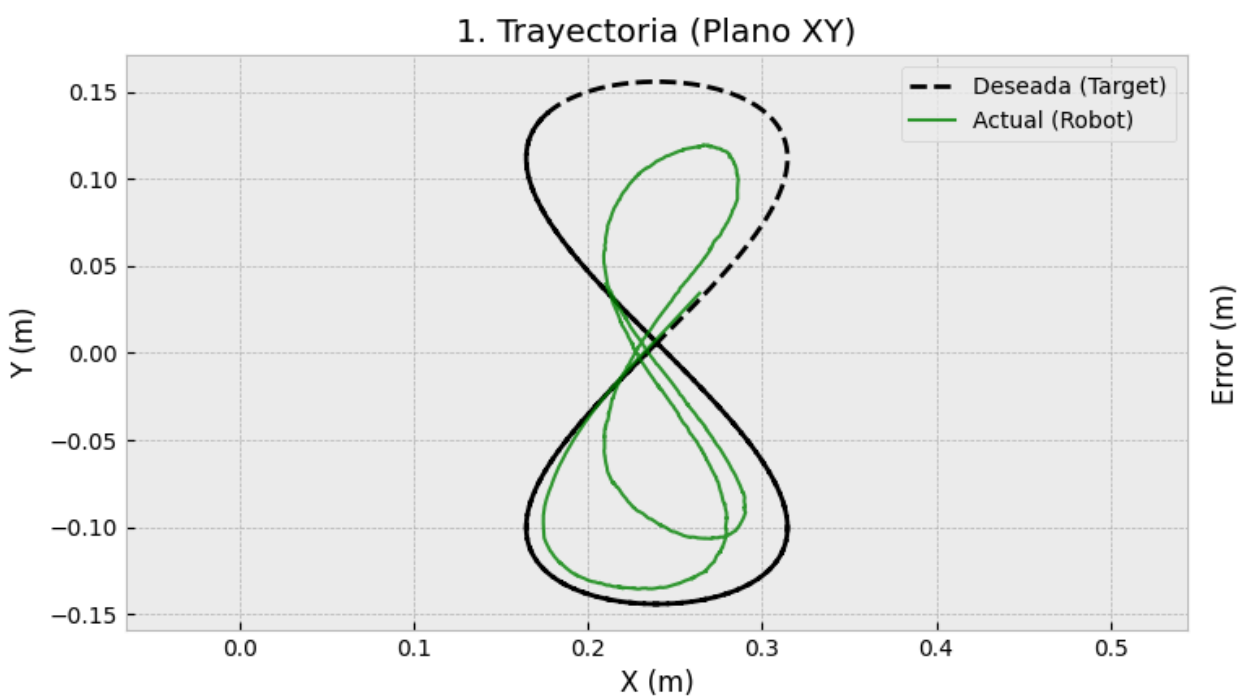
Baseline



Sine

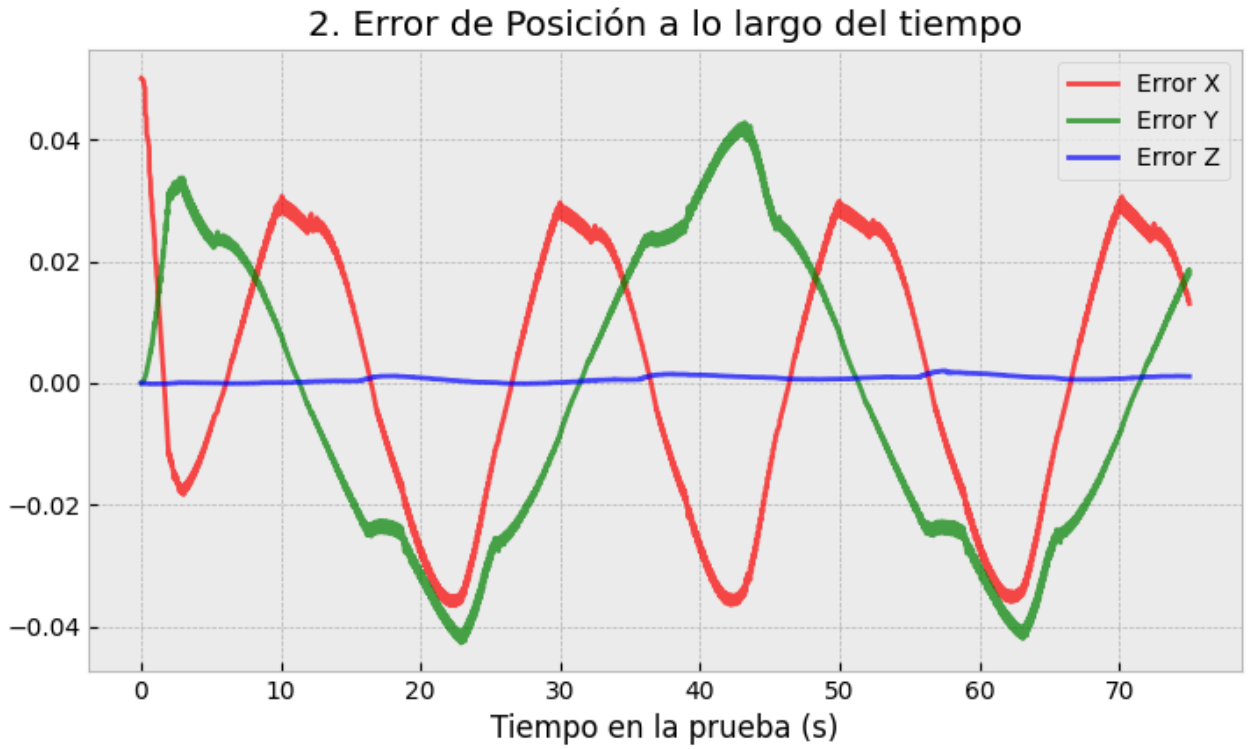


Gaussian

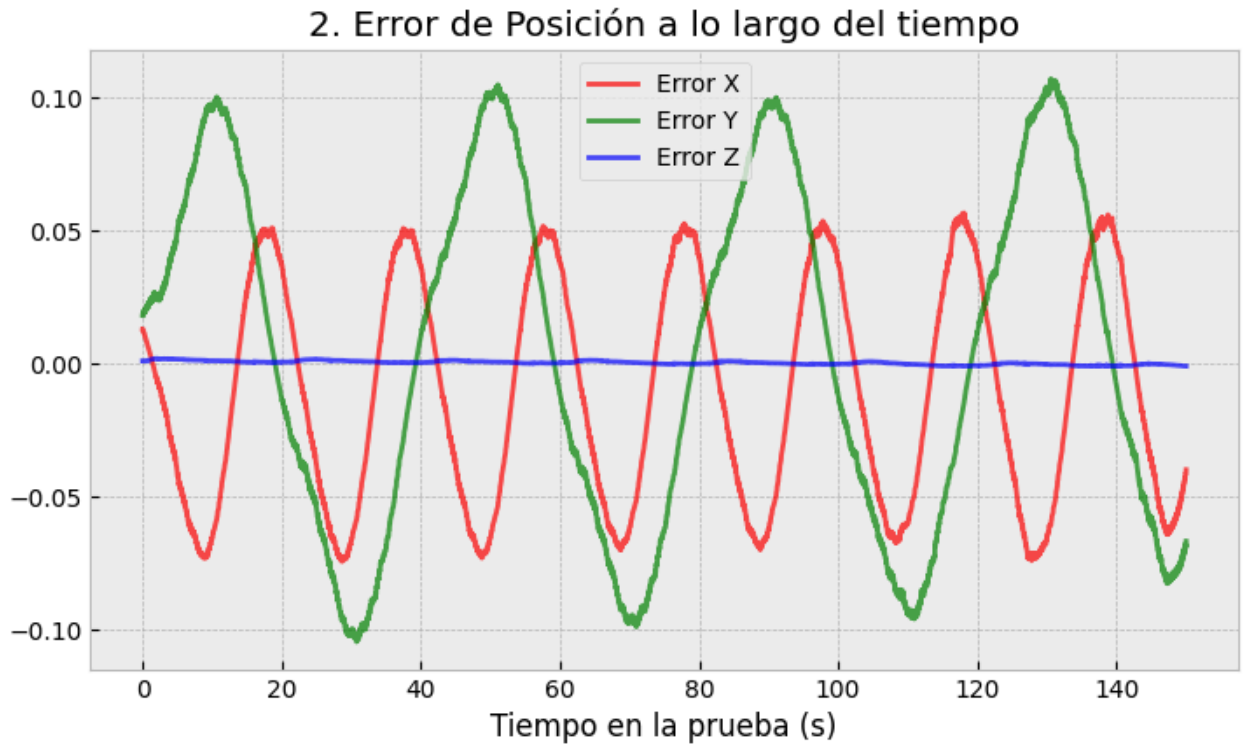


6.2 Error Over Time

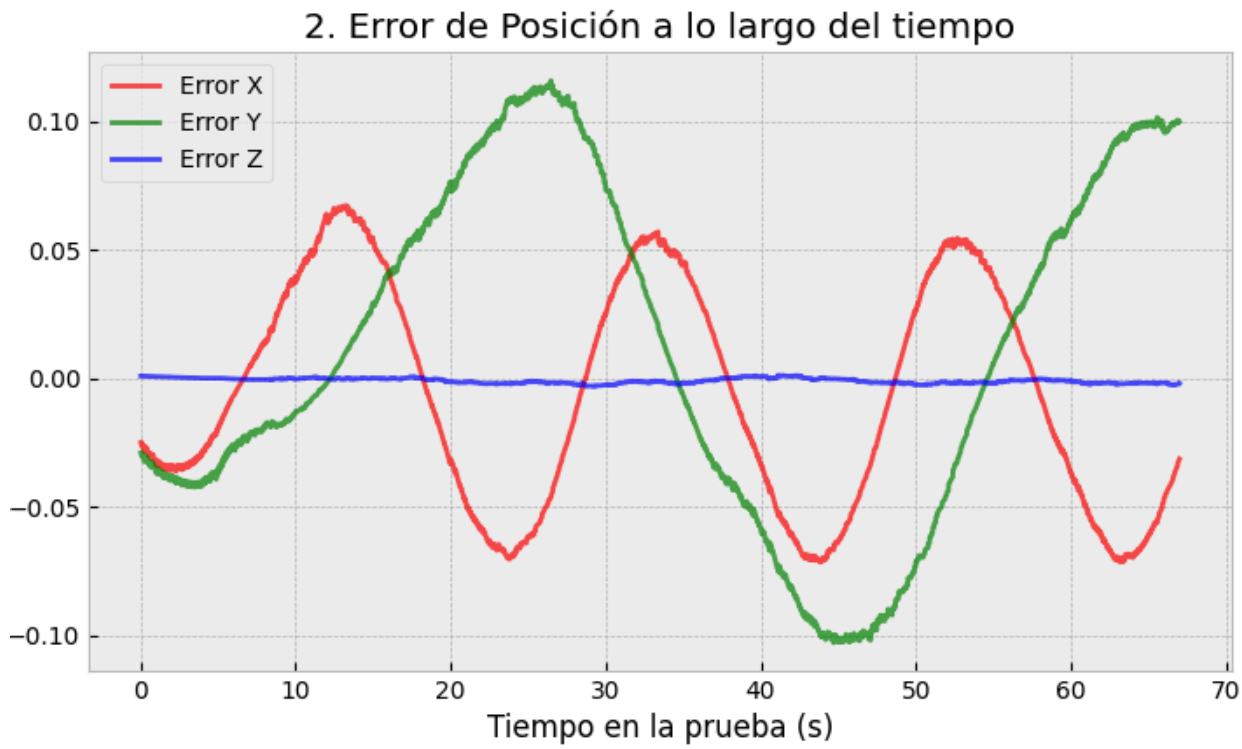
Baseline



Sine

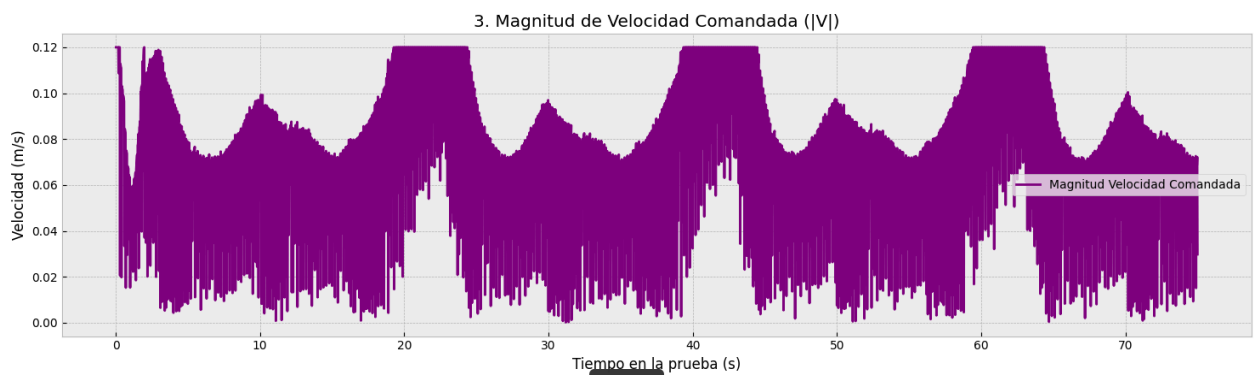


Gaussian

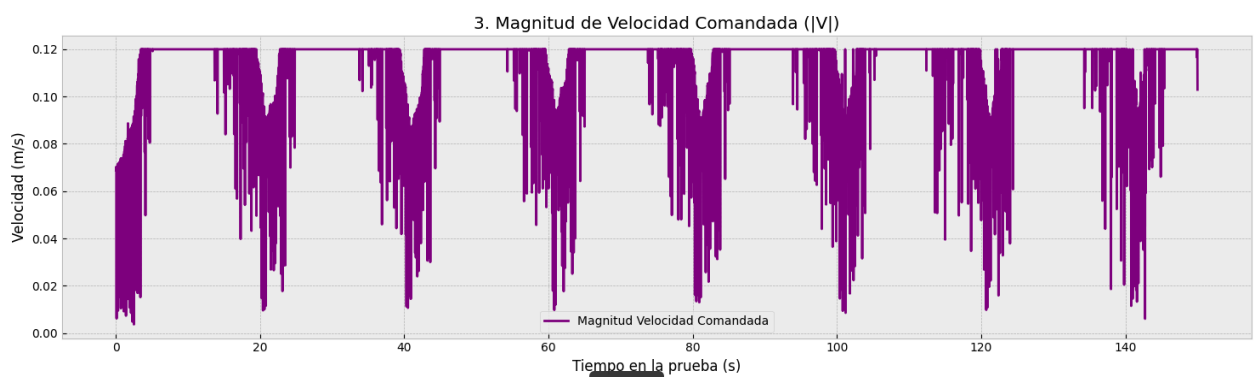


6.3 Commanded Velocity Magnitude

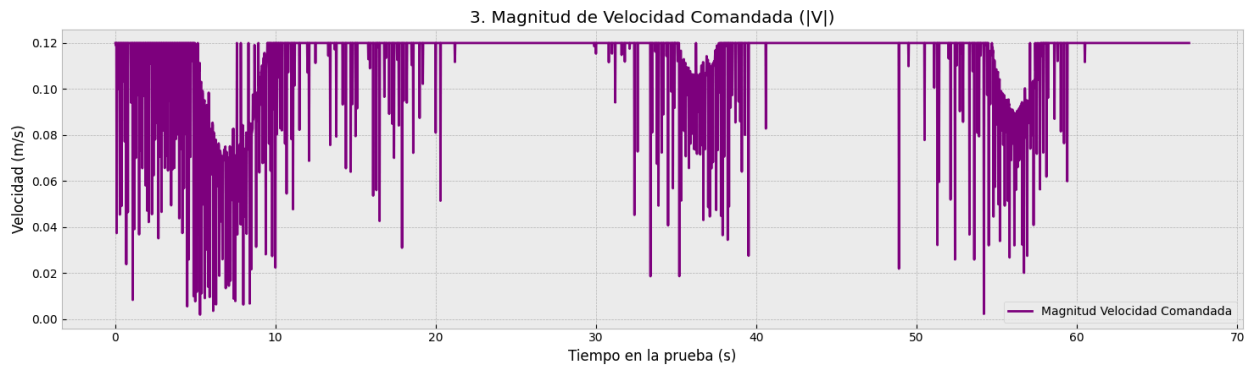
Baseline



Sine



Gaussian

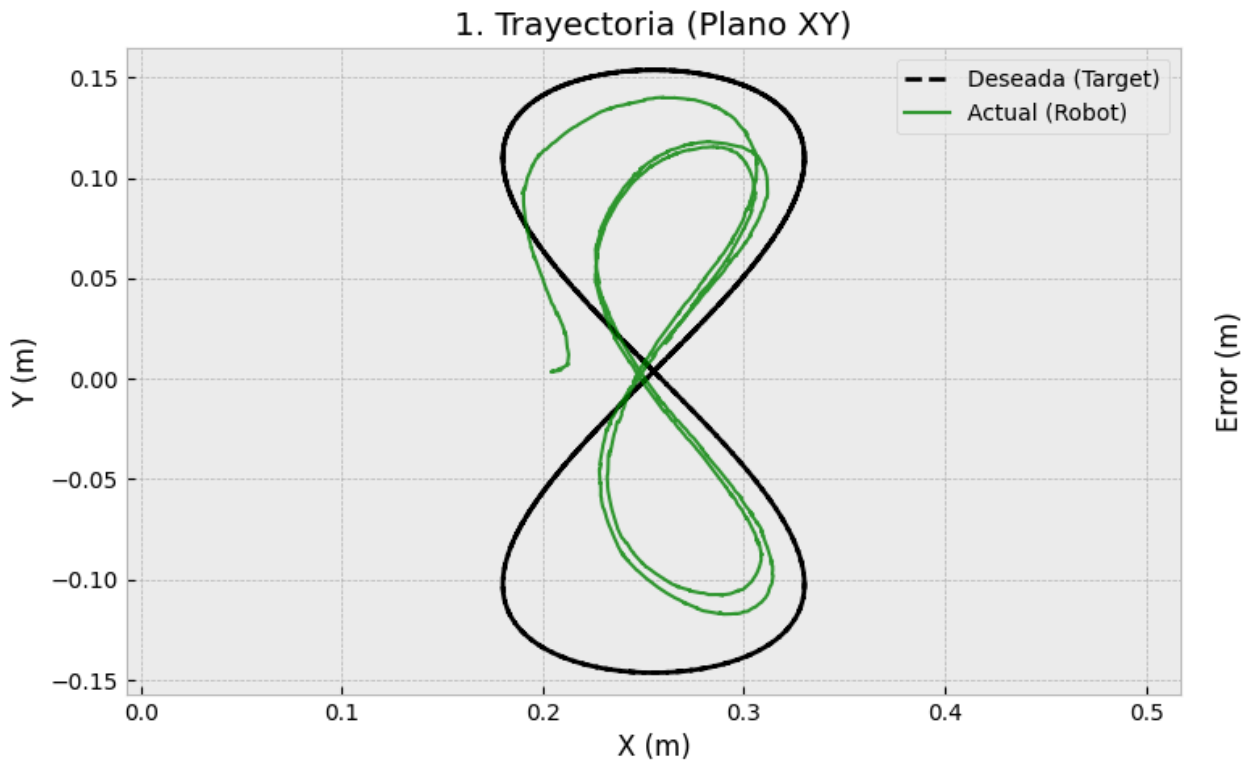


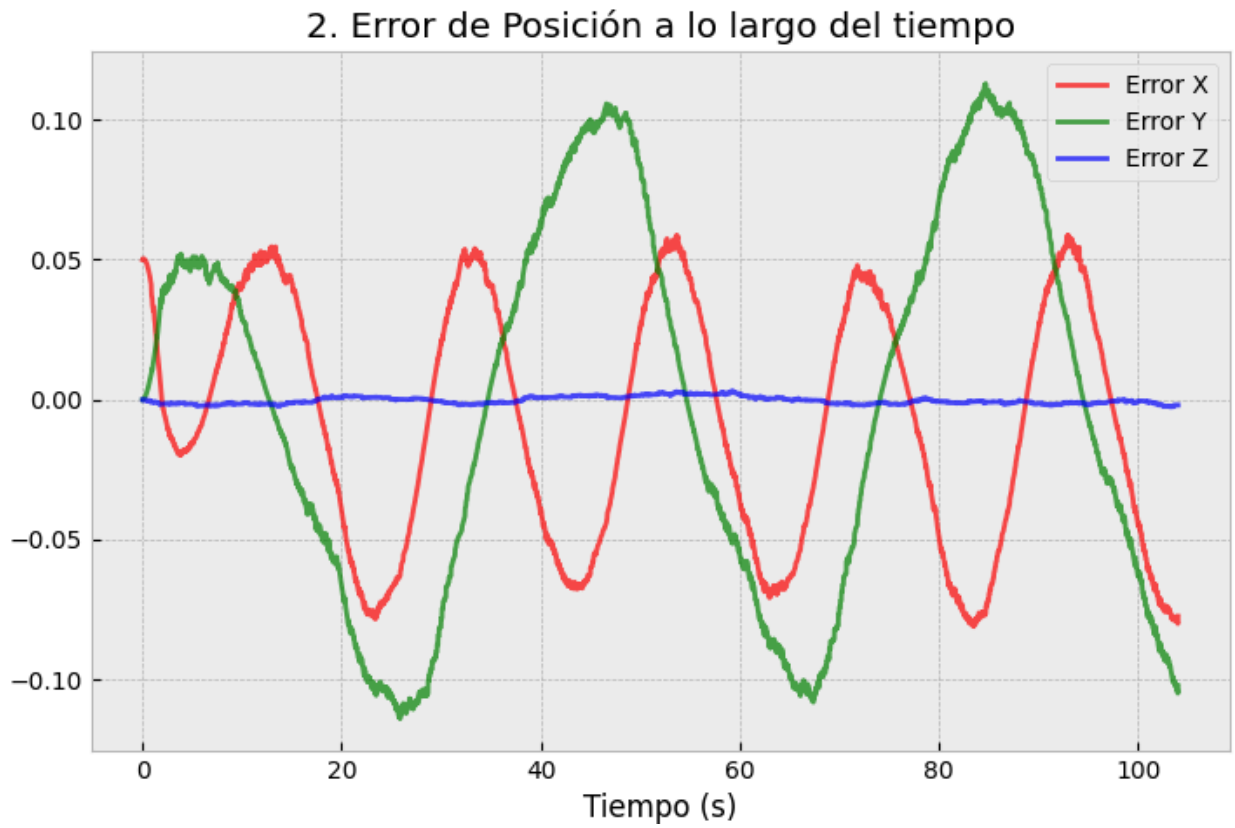
6.4 Quantitative Metrics

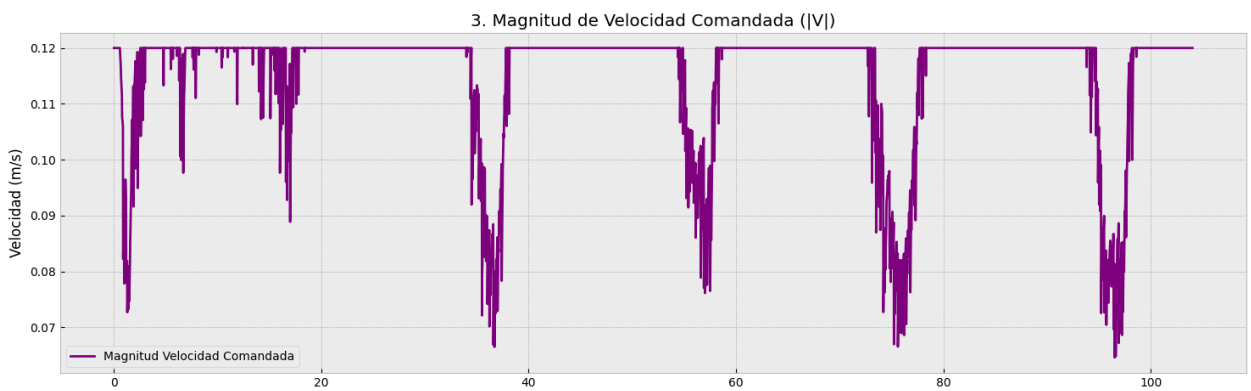
```Shell
==================================================
EXPERIMENTO: BASELINE
==================================================
RMSE X: 0.02196 m
RMSE Y: 0.02377 m
RMSE Z: 0.00085 m
Total RMSE: 0.03238 m
------------------------------------------
Max Error Absoluto X: 0.05000 m
Max Error Absoluto Y: 0.04276 m
Max Error Absoluto Z: 0.00201 m
Max Error Absoluto Global: 0.05000 m

==================================================
EXPERIMENTO: SINE
==================================================
RMSE X: 0.04350 m
RMSE Y: 0.06467 m
RMSE Z: 0.00079 m
Total RMSE: 0.07795 m
------------------------------------------
Max Error Absoluto X: 0.07425 m
Max Error Absoluto Y: 0.10713 m
Max Error Absoluto Z: 0.00201 m
Max Error Absoluto Global: 0.10713 m

==================================================
EXPERIMENTO: GAUSSIAN
==================================================
```

```
RMSE X: 0.04365 m
RMSE Y: 0.06623 m
RMSE Z: 0.00140 m
Total RMSE: 0.07933 m
-----------------------------------------
Max Error Absoluto X: 0.07164 m
Max Error Absoluto Y: 0.11577 m
Max Error Absoluto Z: 0.00319 m
Max Error Absoluto Global: 0.11577 m
```

6.5 Gaussian with filter of first order.







Shell

```
python3 analysis_evaluation2.py
📊 Analizando datos de: baseline_2.csv

================ RESULTADOS DE EVALUACIÓN ================
RMSE X: 0.04425 m
RMSE Y: 0.06790 m
RMSE Z: 0.00130 m
Total RMSE: 0.08106 m
-------------------------------------------
```

```
Max Error Absoluto X: 0.08104 m
Max Error Absoluto Y: 0.11400 m
Max Error Absoluto Z: 0.00316 m
Max Error Absoluto Global: 0.11400 m
==========================================================
```

7. Analysis & Discussion

7.1 Baseline Behavior

The baseline experiment demonstrated stable and accurate tracking, establishing a nominal Total RMSE of 0.03238 m. The tracking on the Z-axis was exceptionally precise (RMSE of 0.00085 m), confirming that the hold-z parameter and the deadband (ϵ=0.002 m) effectively suppressed unnecessary micro-adjustments. The maximum absolute error of 0.05000 m primarily occurred during high-velocity curvature changes where hardware velocity saturation (Vmax=0.12 m/s) temporarily limited the controller's effort.

7.2 Sinusoidal Pertubation – Frequency-Domain

When the sinusoidal perturbation was injected, the Total RMSE more than doubled to 0.07795 m, and the maximum absolute error spiked to 0.10713 m. Because the sine wave injects energy at a discrete, fundamental frequency, the proportional term (Kp) could still respond predictably. Interestingly, the Z-axis error remained completely unaffected (0.00079 m), proving that the disturbance in the XY plane did not destabilize the unperturbed axes.

7.3 Gaussian Pertubation – Noise Sensivitity

The Gaussian perturbation yielded the worst overall performance, with the highest Total RMSE (0.07933 m) and a maximum global error of 0.11577 m. Unlike the deterministic sine wave, Gaussian white noise distributes energy across a broad spectrum of high frequencies. This exposed the inherent noise sensitivity of the derivative term (Kd). The "derivative kick" caused by differentiating the high-frequency stochastic noise led to highly erratic commanded velocities. This erratic behavior even bled into the Z-axis, nearly doubling its RMSE (from 0.00079 m in the sine test to 0.00140 m), demonstrating how high-frequency vibrations in the physical manipulator degrade overall Cartesian precision.

7.4 Stability Explanation

Despite the significant increase in tracking error during the perturbation tests, the system remained bounded-input, bounded-output (BIBO) stable. The physical xArm Lite 6 never entered an emergency halt state, just it started in a pose. This stability is attributed to the careful tuning of the fixed gains and the strict velocity saturation limits.

8. Conclusion

The Cartesian PD controller successfully tracked a continuous parametric trajectory on the physical xArm Lite 6 hardware. The experiments confirmed that while derivative control improves damping and reduces overshoot in nominal conditions, it introduces severe vulnerabilities to high-frequency stochastic noise. To improve performance in noisy real-world environments, a low-pass filter on the error derivative, a best control that PD or PD better implement would be highly recommended.

Github

https://github.com/SoniDavid/control_lab_4.1/tree/main/ros2_ws/src/xarm_perturbations/xarm_perturbations
https://github.com/SoniDavid/control_lab_4.1/tree/main

Anexos

https://drive.google.com/drive/folders/1CWG_GJx56LiHc48TknumAXpezcrNBgoO?usp=sharing